

PELENGKAP DOKUMENTASI ARSITEKTUR v2

Spesifikasi Teknis & Rencana Penguatan Sistem

Database schema, API contracts, error handling, concurrency model, state persistence — serta lima penguatan arsitektur untuk operasional meme coin di hardware terbatas.

Melengkapi: AI Smart Money Trading System — Dokumentasi Arsitektur v2 (Lastico)

Tanggal 21 June 2026

Dokumen ini menambahkan detail teknis tanpa mengubah isi dokumen sumber

Ringkasan Dokumen

Dokumen ini adalah lampiran terpisah, bukan revisi. Tidak ada konten dari dokumen sumber yang diubah atau dihapus — semua di bawah ini adalah penambahan untuk menutup gap menuju tahap development, ditambah rencana penguatan khusus untuk konteks meme coin di hardware terbatas.

Bagian A — Spesifikasi Teknis (Gap Development)

Enam area yang disebutkan secara konseptual di dokumen sumber namun belum punya spesifikasi implementasi.

No	Topik	Menjawab Pertanyaan
A1	Database Schema — closed_trades & watchlist	Tabel apa saja, kolom, tipe data, dan constraint-nya?
A2	API Contracts — RPC, DexScreener, pump.fun	Endpoint mana, format request/response, rate limit, auth?
A3	Error & Recovery Flow	Apa yang terjadi saat RPC putus, API down, atau order gagal?
A4	Konfigurasi Terpusat (Single Source of Truth)	Threshold dan parameter disimpan di mana, bukan tersebar di kode?
A5	Model Konkurensi — 3 Lapis Proteksi Paralel	Paralel itu implementasinya thread, async, atau queue?
A6	State Persistence & Crash Recovery	Kalau laptop restart, apakah posisi & model state pulih otomatis?

Bagian B — Rencana Penguatan untuk Meme Coin di Hardware Terbatas

Lima penambahan yang menempel ke layer arsitektur yang sudah ada, tanpa layer baru dan tanpa biaya tambahan.

No	Penguatan	Layer Terdampak
B1	Dynamic Wallet Discovery (cluster correlation)	Layer 0 — Data Source
B2	Token Age & Liquidity Floor sebagai Hard Filter	Layer 1 — Trigger Engine
B3	Dedup & Cooldown per (wallet, token)	Layer 1 — Trigger Engine
B4	RPC Fallback (Primary/Secondary)	Layer 0 — Data Source
B5	Position Correlation Cap	Layer 3 / Eksekusi Otomatis

Penomoran "Bagian A" dan "Bagian B" digunakan khusus di lampiran ini agar tidak bentrok dengan penomoran 01–06 di dokumen sumber.

Database Schema

Dokumen sumber menyebut kolom-kolom baru (risk_pct, r_multiple, exit_reason, dsb) secara naratif. Berikut struktur tabel lengkap agar bisa langsung dipakai sebagai migration pertama.

Tabel: closed_trades
Satu baris per posisi yang ditutup — sumber utama dataset training (lihat 03 · Pipeline AI, dokumen sumber)

Kolom	Tipe	Keterangan
trade_id	TEXT PK	UUID, primary key
wallet_source	TEXT	'A' atau 'B' — wallet mana yang memicu sinyal
token_address	TEXT	Contract address token, indexed
token_symbol	TEXT	Untuk tampilan dashboard saja, bukan kunci logika
signal_ts	DATETIME	Waktu trigger fire (UTC)
entry_ts	DATETIME	Waktu order entry tereksekusi
exit_ts	DATETIME	Waktu posisi ditutup
direction	TEXT	BUY / SELL — output XGBoost Inference Engine
confidence_score	REAL	0.0–1.0, dari XGBoost (lihat 03 · Pipeline AI)
safety_check_passed	BOOLEAN	Hasil Token Safety Check (lihat 01 · Arsitektur Sistem)
entry_price	REAL	Harga eksekusi aktual
exit_price	REAL	Harga saat posisi ditutup
position_size_usd	REAL	Ukuran posisi dalam USD
risk_pct	REAL	Fixed 1% — disebut di 03 · Pipeline AI & 06 · Eksekusi Otomatis
pnl_pct_actual	REAL	PnL aktual dalam persen
r_multiple	REAL	pnl_pct_actual / risk_pct — basis labeling
label	TEXT	BUY_BENAR / SALAH / HOLD — turunan dari r_multiple
holding_time_minutes	INTEGER	Durasi posisi terbuka
exit_reason	TEXT	SL / trailing_tp / kill_switch_lp / kill_switch_dev_dump / kill_switch_slippage / manual
is_paper_trade	BOOLEAN	Bedakan paper trade vs live, keduanya tetap masuk dataset
model_version	TEXT	Versi model yang menghasilkan sinyal ini (v0, v1, v2, ...)

exit_reason sengaja dipisah dari label (lihat catatan halaman 10 dokumen sumber) — kolom ini yang nanti dipakai untuk query training set bersih: WHERE exit_reason NOT LIKE 'kill_switch%' saat ingin evaluasi kualitas sinyal murni, terpisah dari kejadian rug.

Database Schema — Tabel Pendukung

Dua tabel tambahan yang diperlukan agar wallet monitoring dan model versioning punya tempat penyimpanan eksplisit.

Tabel: watchlist_wallets
Menampung Whale A & B, plus ruang untuk wallet hasil dynamic discovery (lihat B1)

Kolom	Tipe	Keterangan
wallet_address	TEXT PK	Primary key
label	TEXT	'Whale A', 'Whale B', atau label auto-discovery
source	TEXT	'manual' atau 'auto_discovered'
added_at	DATETIME	Kapan masuk watchlist
active	BOOLEAN	Bisa dinonaktifkan tanpa hapus histori

Tabel: model_registry
Mencatat setiap retrain — diperlukan untuk rollback guard (lihat 03 · Pipeline AI)

Kolom	Tipe	Keterangan
model_version	TEXT PK	v0, v1, v2, ...
trained_at	DATETIME	Waktu training selesai
training_sample_count	INTEGER	Jumlah closed trades yang dipakai
validation_accuracy	REAL	Untuk perbandingan rollback guard
expectancy_r	REAL	Metrik utama, lihat 03 · Pipeline AI
is_active	BOOLEAN	Hanya satu baris boleh TRUE pada satu waktu
rolled_back	BOOLEAN	TRUE jika model ini pernah di-rollback otomatis

Catatan implementasi: untuk hardware terbatas, SQLite sudah cukup untuk seluruh schema ini — tidak perlu Postgres/MySQL terpisah. Volume data (puluhan-ratusan closed trades per 30 hari) jauh di bawah ambang di mana SQLite jadi bottleneck.

API Contracts

Dokumen sumber menyebut "RPC/Indexer", "DexScreener", dan "pump.fun API" sebagai sumber data. Berikut kontrak minimal tiap integrasi: apa yang dipanggil, apa yang diharapkan kembali, dan batasannya.

1. Blockchain RPC / Indexer

Layer 0

Sumber event on-chain: swap, transfer, LP change dari Whale A & B

Aspek	Detail
Metode koneksi	WebSocket subscription (logsSubscribe / accountSubscribe) — bukan polling, sesuai kebutuhan window 5 menit yang responsif
Rate limit (tier gratis)	RPC publik umumnya membatasi request/detik dan jumlah koneksi WS bersamaan — perlu dicek di provider yang dipilih sebelum implementasi
Data yang dibutuhkan	Signature transaksi, instruction type, token mint, jumlah, wallet pengirim/penerima
Failure mode	Koneksi WS terputus tanpa pesan eksplisit — perlu heartbeat/ping internal untuk deteksi dini (lihat A3)

2. DexScreener / Block Explorer API

Layer 2/3

Sumber Token Safety Check: liquidity lock, kontrak verified, holder distribution

Aspek	Detail
Metode koneksi	REST request per token saat dibutuhkan (bukan polling kontinu) — dipanggil hanya setelah XGBoost Inference menghasilkan output
Rate limit (tier gratis)	API publik biasanya membatasi request/menit — karena dipanggil per-sinyal (bukan per-detik), risiko rate limit relatif rendah, tapi tetap perlu caching singkat per token untuk hindari panggilan berulang dalam window pendek
Data yang dibutuhkan	Liquidity locked status, contract verification flag, top-10 holder percentage, mint authority status
Failure mode	API timeout/down → safety check tidak bisa dievaluasi; default behaviour harus fail-closed (alert diblokir), bukan fail-open

3. pump.fun API

Eksekusi Otomatis

Auto-entry, market order exit, dan quote untuk deteksi slippage (kill-switch Lapis 3)

Aspek	Detail
Metode koneksi	REST untuk order placement; polling quote singkat untuk deteksi slippage melonjak (kill-switch Lapis 3)
Auth	Wallet keypair lokal untuk signing transaksi — private key tidak boleh keluar dari mesin lokal (lihat catatan keamanan di bawah)
Data yang dibutuhkan	Quote (harga + slippage estimate), order confirmation, transaction status
Failure mode	Order gagal/timeout saat exit kill-switch — skenario paling kritis, perlu retry-immediate dengan slippage tolerance dinaikkan bertahap (lihat A3)

Catatan keamanan: dokumen sumber maupun lampiran ini tidak membahas custody private key secara eksplisit. Private key wallet trading harus disimpan terenkripsi secara lokal (bukan plaintext di file config) dan idealnya terpisah dari wallet utama — wallet trading sebaiknya hanya diisi modal yang memang dialokasikan untuk sistem ini.

Error & Recovery Flow

Tidak dibahas sama sekali di dokumen sumber. Untuk sistem yang memegang posisi terbuka dengan uang riil, ini adalah gap paling kritis untuk ditutup sebelum development eksekusi otomatis dimulai.

Skenario Kegagalan	Dampak Jika Tidak Ditangani	Perilaku yang Diharapkan
RPC WebSocket terputus	Sistem "buta" terhadap pergerakan wallet — bisa miss sinyal entry, dan lebih bahaya: miss sinyal kill-switch untuk posisi yang sedang terbuka	Reconnect otomatis dengan exponential backoff; jika reconnect gagal > N kali, alihkan ke RPC fallback (lihat B4) dan kirim notifikasi lokal
DexScreener/safety API timeout	Token Safety Check tidak bisa dievaluasi	Fail-closed: alert diblokir, sinyal tetap di-log untuk training (konsisten dengan filosofi 03 · Pipeline AI), retry sekali sebelum dianggap gagal
pump.fun order entry gagal	Sinyal valid terlewat tanpa posisi terbuka	Retry maksimal 2x dalam <5 detik; jika tetap gagal, log sebagai "missed_entry" — <i>jangan</i> ikut campur ke dataset closed_trades karena posisi tidak pernah benar-benar terbuka
pump.fun order exit gagal (kondisi normal)	Posisi tetap terbuka melebihi SL/TP yang seharusnya	Retry langsung dengan slippage tolerance dinaikkan bertahap; alert ke dashboard sebagai "exit pending" sampai konfirmasi diterima
pump.fun order exit gagal (saat kill-switch trigger)	Skenario terburuk — rug terdeteksi tapi posisi tidak bisa keluar	Retry agresif tanpa jeda dengan slippage tolerance maksimal; ini satu-satunya kasus di mana "rugi ke slippage demi keluar" diprioritaskan mutlak di atas segalanya
Proses sistem crash saat posisi terbuka	Tiga lapis proteksi paralel berhenti memonitor — posisi "telanjang" tanpa SL/TP/kill-switch aktif	Lihat A6 — state persistence wajib agar posisi ter-restore begitu proses restart, idealnya dengan watchdog terpisah yang restart proses otomatis
Retrain cron terlewat (laptop mati/sleep saat 02:00 UTC)	Model tidak ter-update, sistem terus pakai model lama tanpa disadari	Saat startup, cek timestamp retrain terakhir di model_registry — jika > 24 jam, jalankan retrain segera alih-alih menunggu jadwal berikutnya

Prinsip Umum Recovery

- **Fail-closed untuk keputusan trading** — saat ragu (data tidak lengkap, API tidak merespons), default-nya selalu "jangan kirim alert / jangan buka posisi", bukan sebaliknya.
- **Fail-open hanya untuk monitoring posisi yang sudah terbuka** — sistem harus tetap mencoba keluar dari posisi sebisa mungkin, karena diam berarti risiko terus berjalan.
- **Semua kegagalan tercatat dengan timestamp dan konteks** — bukan sekadar log teks, tapi entri terstruktur yang bisa dikorelasikan dengan exit_reason di closed_trades.

Konfigurasi Terpusat

Angka-angka seperti confidence threshold 75%, trigger window 5 menit, dan tier trailing TP tersebar sebagai teks naratif di dokumen sumber. Berikut satu struktur config sebagai single source of truth.

```
# config.yaml – satu file, semua threshold operasional

trigger_engine:
  window_minutes: 5
  mode: "AND"           # AND | OR – lihat 01 · Arsitektur Sistem

decision_gate:
  confidence_threshold: 0.75

risk:
  risk_pct_per_trade: 0.01

trailing_tp:
  tiers:
    - {r_min: 1, r_max: 2, trail_pct: null}  # SL tetap -1R
    - {r_min: 2, r_max: 5, trail_pct: 0.25}
    - {r_min: 5, r_max: 10, trail_pct: 0.15}
    - {r_min: 10, r_max: null, trail_pct: 0.10}

labeling:
  buy_benar_threshold_r: 3.0
  salah_threshold_r: -1.0

retrain:
  schedule_utc: "02:00"
  min_closed_trades_first: 100
  min_closed_trades_alt: 50
  min_buy_benar_in_alt: 15
  rolling_window_days: 30
  rollback_accuracy_drop_pct: 0.05

kill_switch:
  dev_wallet_sell_threshold_pct: # % dari supply – perlu dikalibrasi
  slippage_spike_threshold_pct:  # perlu dikalibrasi
```

Mengapa ini penting di tahap awal

- Dokumen sumber sendiri menyebut beberapa angka sebagai *"starting point, bukan angka final"* (lihat halaman 11) — config terpusat membuat kalibrasi ulang cukup ubah satu file, tanpa menyentuh kode.
- Memisahkan parameter dari logika memudahkan backtesting dengan kombinasi threshold berbeda tanpa redeploy.
- Kolom `dev_wallet_sell_threshold_pct` dan `slippage_spike_threshold_pct` sengaja dikosongkan — keduanya disebut di 06 · Eksekusi Otomatis tanpa angka final, perlu dikalibrasi dari data live sebelum diisi.

Model Konkurensi

Dokumen sumber (06 · Eksekusi Otomatis) menyatakan tiga lapis proteksi "berjalan paralel" secara konseptual. Berikut pemetaan ke model implementasi konkret yang ringan untuk laptop biasa.

Komponen	Model Konkurensi	Alasan
Lapis 1 — Price-based SL	Async task, evaluasi tiap kali harga update	Event-driven dari price feed, tidak perlu thread terpisah — cukup callback ringan
Lapis 2 — Staged trailing TP	Async task, evaluasi tiap kali harga update	Sama seperti Lapis 1, bisa berbagi event loop yang sama karena logikanya non-blocking
Lapis 3 — Kill-switch on-chain	Thread/process terpisah , event-driven via WebSocket RPC	Disebut eksplisit di dokumen sumber sebagai "proses/thread terpisah dengan prioritas tertinggi" — perlu isolasi agar tidak terblokir oleh evaluasi Lapis 1/2
Trigger engine (Layer 1) & ML pipeline (Layer 2)	Async queue tunggal	Tidak butuh paralelisme tinggi — volume sinyal dari 2 wallet jauh di bawah kebutuhan multi-threading

Mengapa bukan multiprocessing penuh

- Untuk laptop biasa/gaming tanpa server dedicated, satu event loop async (mis. Python asyncio) + satu thread terpisah untuk kill-switch sudah cukup — menghindari overhead context-switching dari banyak proses.
- XGBoost inference (Layer 2) adalah operasi singkat (milidetik untuk model sekecil ini) — tidak perlu proses terpisah, cukup dipanggil sinkron di dalam async task.
- Retrain (Layer 4) adalah satu-satunya operasi yang layak dijalankan sebagai proses terpisah, karena durasinya lebih lama (dokumen sumber menyebut target <2 menit) dan sebaiknya tidak memblokir monitoring real-time selama berjalan.

Titik kritis: karena Lapis 3 punya prioritas tertinggi dan harus override Lapis 1/2 (lihat halaman 10 dokumen sumber), komunikasi antar thread/proses perlu mekanisme sinyal yang langsung diperiksa sebelum setiap evaluasi SL/TP — bukan menunggu giliran di antrian yang sama dengan tugas lain.

State Persistence & Crash Recovery

Tidak dibahas di dokumen sumber. Untuk sistem yang berjalan di laptop biasa (bukan server dengan uptime terjamin), ini menentukan apakah posisi terbuka "selamat" saat restart tak terduga.

Risiko Tanpa State Persistence

Laptop sleep, update OS, atau proses crash saat posisi sedang terbuka → begitu sistem hidup lagi, ia tidak tahu ada posisi yang masih perlu dimonitor. Tiga lapis proteksi (06 · Eksekusi Otomatis) berhenti total selama gap ini.

Apa yang Wajib Di-persist (bukan hanya di memory)

- 1 **Posisi terbuka saat ini** — token, entry price, size, SL level, trailing TP state (tier R saat ini), waktu entry. Ditulis ke disk *segera setelah* entry order terkonfirmasi, bukan di-batch.
- 2 **Model aktif & versinya** — model_version mana yang sedang dipakai (lihat tabel model_registry, A1), agar tidak perlu retrain ulang dari nol saat restart.
- 3 **Watchlist wallet** — terutama jika dynamic discovery (lihat B1) aktif, daftar wallet hasil auto-discovery tidak boleh hilang saat restart.
- 4 **Timestamp retrain terakhir** — untuk logika catch-up retrain yang disebut di A3.
- 5 **Cooldown state per (wallet, token)** — lihat B3, agar restart tidak menyebabkan alert duplikat untuk sinyal yang baru saja diproses sebelum crash.

Alur Saat Startup

- 1 Baca tabel open_positions dari SQLite — jika ada baris dengan status "open", langsung re-aktifkan tiga lapis proteksi untuk posisi tersebut **sebelum** mulai monitoring wallet baru.
- 2 Verifikasi harga & status token saat ini terhadap level SL/TP yang tersimpan — jika ternyata harga sudah melewati SL selama sistem mati, segera proses exit market order.
- 3 Cek timestamp retrain terakhir vs sekarang — jalankan retrain catch-up jika perlu (lihat A3).
- 4 Muat watchlist wallet & model aktif dari database, baru lanjut ke monitoring normal (kembali ke Layer 0).

Implementasi minimal: tabel open_positions di SQLite yang sama dengan closed_trades (lihat A1), dengan baris dipindah ke closed_trades begitu posisi ditutup. Tidak perlu sistem state-management terpisah untuk skala ini.

Penguatan untuk Konteks Meme Coin

Lima penambahan berikut menempel ke layer arsitektur yang sudah ada di dokumen sumber — tidak ada layer baru, tidak ada biaya tambahan, dan semuanya tetap ringan untuk laptop biasa/gaming tanpa GPU. Mengikuti pola "05 · Rencana Peningkatan" di dokumen sumber, tiap item dipetakan ke layer spesifik.

B1 · LAYER 0

Dynamic Wallet Discovery

Watchlist tidak lagi statis 2 wallet — sistem usul wallet baru yang konsisten profit dari token yang sama.

B2 · LAYER 1

Token Age & Liquidity Floor

Hard filter sebelum masuk pipeline ML, bukan sekadar fitur input — mengurangi noise di sumbernya.

B3 · LAYER 1

Dedup & Cooldown

Mencegah alert berulang dari satu kejadian whale yang sama pada token yang sama.

B4 · LAYER 0

RPC Fallback

Single point of failure pada satu RPC publik diatasi dengan primary/secondary tanpa biaya.

B5 · EKSEKUSI

Position Correlation Cap

Mencegah exposure menumpuk saat banyak sinyal berkorelasi muncul bersamaan (umum di meme season).

CATATAN

Bukan Saran Strategi

Lima item ini murni penguatan robustness sistem — bukan rekomendasi entry/exit atau target profit.

Total biaya implementasi: Rp 0

Konsisten dengan filosofi "05 · Rencana Peningkatan" di dokumen sumber — semua memakai data publik gratis dan logika tambahan, bukan layanan berbayar baru.

Dynamic Wallet Discovery

Layer terdampak: **Layer 0** — Data Source & Wallet Monitoring

Masalah

Watchlist saat ini statis: hanya Whale A & B (lihat 01 · Arsitektur Sistem). Di ekosistem meme coin, "smart money" sering berupa cluster wallet yang berubah-ubah — wallet baru yang konsisten profit di token yang sama dengan wallet existing tidak akan pernah terdeteksi oleh sistem yang watchlist-nya tetap.

Mekanisme

Baru

Murni rule-based, berjalan sebagai proses ringan terpisah dari real-time pipeline

- 1

Setiap kali Whale A/B trading sebuah token, catat wallet lain mana saja yang juga masuk ke token yang sama dalam window waktu berdekatan (mis. ± 10 menit).
- 2

Jika satu wallet eksternal muncul berulang (mis. ≥ 3 kali) pada token-token yang sama dengan watchlist existing, **dan** exit dengan profit (verifikasi lewat closed-price pasca-exit dari data publik), tandai sebagai kandidat.
- 3

Kandidat masuk watchlist_wallets (lihat A1) dengan source = 'auto_discovered' — **tidak langsung otomatis aktif**, tampil di dashboard untuk approval manual dulu sebelum dianggap setara Whale A/B.

Kenapa ada approval manual: auto-discovery murni statistik bisa salah tangkap wallet yang kebetulan profit sekali dua kali, bukan benar-benar "smart money". Step approval ini murni desain robustness — keputusan akhir tetap di tangan Anda, bukan otomatis penuh.

Biaya komputasi: *minimal* — ini query agregat sederhana terhadap data on-chain yang sudah ditarik untuk Layer 0, tidak menambah beban RPC baru karena memanfaatkan data transaksi yang sama.

Token Age & Liquidity Floor sebagai Hard Filter

Layer terdampak: **Layer 1 — Trigger Engine** (sebelum Rule-Based Classifier meneruskan ke trigger window)

Masalah

Saat ini "token age" hanya muncul sebagai input feature untuk XGBoost (lihat 03 · Pipeline AI, kelompok "Historical wallet" → "Past exit pattern"). Token yang baru launch dalam hitungan menit punya volatilitas dan noise ekstrem yang berbeda karakter dari token yang sudah establish — membiarkan model "belajar" dari kasus ekstrem ini lewat feature saja berisiko mencemari training data, bukan membantunya.

Mekanisme Baru

Filter keras di Layer 1, sebelum event sempat masuk trigger window — selaras dengan posisi Rule-Based Classifier di 01 · Arsitektur Sistem

Kondisi	Perlakuan
Token age di bawah ambang minimum	Event diabaikan sebelum trigger window — tidak masuk dataset training sama sekali (beda dari sinyal confidence rendah yang tetap di-log)
Liquidity pool depth di bawah ambang minimum	Sama — diabaikan di Layer 1, karena pool dangkal membuat slippage tidak representatif untuk evaluasi sinyal apapun
Token age & liquidity di atas ambang	Lanjut normal ke trigger window seperti alur existing di dokumen sumber

Angka ambang minimum perlu dikalibrasi dari data live (sama seperti threshold lain di dokumen sumber yang ditandai "starting point, bukan angka final") — tidak ditetapkan di sini karena bergantung pada karakteristik wallet target yang dipantau.

Beda dengan Rule-Based Classifier yang sudah ada: Rule-Based Classifier (01 · Arsitektur Sistem) memfilter *jenis aktivitas* (swap relevan vs transfer internal). Filter ini memfilter *kondisi token* itu sendiri — keduanya saling melengkapi di titik yang sama (Layer 0/1), bukan saling menggantikan.

Dedup & Cooldown per (Wallet, Token)

Layer terdampak: **Layer 1 — Trigger Engine**

Masalah

Kalau whale yang sama melakukan beberapa transaksi pada token yang sama dalam window pendek (umum saat whale membangun posisi bertahap), sistem berisiko fire alert berulang untuk apa yang sebenarnya satu keputusan trading, bukan beberapa sinyal independen — menyebabkan alert fatigue di dashboard dan duplikasi di dataset training.

Mekanisme

Baru

Cooldown key sederhana, dicek setelah Filter Relevansi dan sebelum Trigger Window (lihat 02 · Flow Aplikasi)

- 1 Setiap event yang lolos Filter Relevansi dicek terhadap cooldown table dengan key (wallet_address, token_address).
- 2 Jika pasangan ini sudah memicu trigger dalam window cooldown (terpisah dari trigger window 5 menit yang sudah ada — cooldown ini lebih panjang, mis. per posisi aktif), event baru diabaikan untuk tujuan trigger baru, namun datanya tetap bisa dipakai memperbarui posisi yang sudah dipantau.
- 3 Cooldown otomatis reset begitu posisi terkait token tersebut closed (lihat closed_trades, A1) — sehingga whale yang sama boleh memicu sinyal baru lagi untuk token yang sama di kesempatan trading berikutnya.

Implementasi: satu tabel kecil di SQLite (key + timestamp), dicek dengan satu query sebelum event diteruskan — overhead nyaris nol, dan persis kebutuhan A6 (state persistence) untuk cooldown state agar tidak hilang saat restart.

RPC Fallback (Primary / Secondary)

Layer terdampak: Layer 0 — Data Source & Wallet Monitoring

Masalah

Sistem dirancang berjalan 24/7 di laptop dengan RPC publik gratis. Bergantung pada satu provider RPC saja membuat seluruh sistem berhenti total (termasuk monitoring kill-switch untuk posisi terbuka) setiap kali provider tersebut mengalami downtime atau rate limit — risiko operasional yang signifikan untuk komponen yang justru paling kritis.

Mekanisme

Baru

Dua RPC publik gratis (primary + secondary), bukan biaya tambahan — hanya logika switching

Kondisi	Perilaku
RPC primary normal	Semua traffic lewat primary, seperti alur existing
RPC primary gagal reconnect (lihat A3)	Alihkan otomatis ke RPC secondary, catat event switchover untuk visibilitas di dashboard
RPC secondary juga gagal	Sistem masuk mode "degraded" — hentikan trigger baru, tapi tetap coba pantau posisi terbuka lewat polling fallback (lebih lambat dari WebSocket, tapi lebih baik dari buta total)
Kedua RPC pulih	Kembali ke primary, log durasi downtime untuk evaluasi keandalan provider

Ini melengkapi A3 (Error & Recovery Flow) — A3 mendefinisikan apa yang terjadi saat RPC gagal, B4 mendefinisikan ke mana sistem beralih.

Position Correlation Cap

Layer terdampak: **Eksekusi Otomatis** (dievaluasi sebelum auto-entry, lihat 06 · Eksekusi Otomatis)

Masalah

Setiap posisi dirancang independen dengan risk 1% per trade (06 · Eksekusi Otomatis) — perhitungan ini valid *per posisi*, tapi mengasumsikan posisi-posisi tersebut tidak berkorelasi. Di musim meme coin yang sedang ramai, beberapa sinyal dengan token berkorelasi tinggi (mis. sama-sama mengikuti tren naratif yang sama) bisa muncul nyaris bersamaan — total exposure riil terhadap satu "risiko bersama" bisa jauh melebihi asumsi $1\% \times \text{jumlah posisi}$.

Mekanisme

Baru

Satu pengecekan tambahan sebelum auto-entry, paralel dengan Token Safety Check yang sudah ada

- 1 Tentukan batas jumlah posisi terbuka bersamaan secara keseluruhan (mis. maksimal N posisi aktif) — angka spesifik adalah keputusan risk tolerance Anda sendiri, bukan sesuatu yang bisa ditentukan dari sisi arsitektur.
- 2 Sebelum auto-entry baru dieksekusi, cek jumlah baris di `open_positions` (lihat A6) — jika sudah di batas, sinyal baru tetap di-log (konsisten dengan filosofi "tidak ada sinyal dibuang" di dokumen sumber) tapi tidak dieksekusi.
- 3 Opsional lanjutan: kelompokkan posisi berdasarkan kemiripan waktu launch token atau sumber wallet trigger, agar cap bisa lebih spesifik dari sekadar "jumlah total" — ini pengembangan lanjutan, bukan kebutuhan v1.

Ini bukan saran soal angka berapa N idealnya. Penentuan batas eksposur adalah keputusan manajemen risiko pribadi yang bergantung pada modal dan toleransi risiko Anda — bagian arsitektur ini hanya menyediakan *mekanisme* agar batas tersebut benar-benar ditegakkan oleh sistem, bukan terlewat begitu saja saat banyak sinyal datang bersamaan.

Checklist Kesiapan Development

Ringkasan status setelah lampiran ini melengkapi dokumen sumber.

Area	Status	Sumber
Arsitektur 5-layer & flow keputusan	✓ Lengkap	Dokumen sumber, 01–02
Feature engineering & konfigurasi model	✓ Lengkap	Dokumen sumber, 03
Dataset & labeling strategy (R-multiple)	✓ Lengkap	Dokumen sumber, 03
Tiga lapis proteksi eksekusi	✓ Lengkap	Dokumen sumber, 06
Database schema	✓ Lengkap	Lampiran, A1
API contracts	✓ Lengkap	Lampiran, A2
Error & recovery flow	✓ Lengkap	Lampiran, A3
Konfigurasi terpusat	✓ Lengkap	Lampiran, A4
Model konkurensi	✓ Lengkap	Lampiran, A5
State persistence & crash recovery	✓ Lengkap	Lampiran, A6
Private key custody	⚠ Disinggung, belum detail	Lampiran, A2 (catatan keamanan)
Angka threshold final (trail %, kill-switch %)	⚠ Disengaja kosong	Perlu kalibrasi dari data live

Yang masih perlu keputusan Anda sebelum coding dimulai

- Pilihan provider RPC primary & secondary (B4) yang sesuai rate limit kebutuhan Anda
- Mekanisme penyimpanan private key wallet trading (lihat catatan keamanan, A2)
- Batas jumlah posisi terbuka bersamaan / N pada Position Correlation Cap (B5)
- Apakah Dynamic Wallet Discovery (B1) diaktifkan sejak v1, atau ditunda ke v2 setelah sistem inti stabil

Dokumen ini adalah lampiran independen. Seluruh isi dokumen sumber (AI Smart Money Trading System — Dokumentasi Arsitektur v2) tetap berlaku tanpa perubahan; lampiran ini hanya menambahkan detail implementasi dan penguatan robustness yang belum tercakup di sana.